分布式计算-实验 2

黎俊
241275043

2026 年 10 月 25 日

目录

1 实验概述 2

1.1 实验目的 2

1.2 实验环境 2

1.3 数据集 2

2 设计思路 3

2.1 任务 1：各开户机构客户数量及资产统计 3

2.2 任务 2：平均值分析 3

2.3 任务 3：客户总资产四分位数 4

2.4 任务 4：客户资产分层与资产结构统计 4

2.5 任务 5：客户信贷风险预测（高斯朴素贝叶斯） 4

3 运行结果 5

3.1 任务 1 结果 5

3.2 任务 2 结果 5

3.3 任务 3 结果 5

3.4 任务 4 结果 6

3.5 任务 5 结果 7

4 可能的改进之处 7

附录：运行成功的 Web 页面截图 8

1 实验概述

1.1 实验目的

本实验基于 Hadoop MapReduce 分布式计算框架，对银行脱敏信贷客户数据进行统计分析与建模预测。实验共 5 个任务：任务 1–4 对原始客户数据做特征工程层面的预处理与统计探索分析，挖掘客户资产分布、分群特征；任务 5 在前期分析基础上利用 MapReduce 实现客户信贷风险预测模型，理解分布式计算框架下机器学习预测任务的实现思路。

通过本实验重点掌握 MapReduce 编程范式，学习分布式缓存（DistributedCache）、Mapper/Reducer 分工、任务调度、模型加载与分布式预测等核心知识。

1.2 实验环境

表 1: 实验环境配置

组件	版本	说明
OS	Ubuntu	64 位
Java	OpenJDK 11	Hadoop 3.x 要求
Hadoop	3.5.0	单机伪分布式
Maven	3.9.9	项目构建

1.3 数据集

数据来源为脱敏后的银行信贷客户数据，原始数据含 627 个字段，经筛选保留 15 个有效字段，分为训练集 `train.csv` 与测试集 `test.csv`。任务 1–4 只使用 `train.csv`，任务 5 同时使用两个数据集。主要字段如下：

表 2: 主要字段说明

列名	类型	含义
CUST_ID	bigint	客户唯一标识 ID
OPEN_ORG_NUM	bigint	开户机构编号
bad_good	bigint	客户好坏标签（0/1）
CUST_SAVING_AMT ~ CUST_METAL_AMT	double	存款、理财、私人理财、基金、国债、
CUST_ASSET_AMT	double	资产总额
CUST_DEBT_AMT	double	负债总额
G_OS_PRCP_SUM	double	贷款账户月余额
LOAN_FLAG / CRED_FLAG	string	个贷标识 / 持有基金标志

其中训练集含 285285 条记录（好客户 280401，坏客户 4884），测试集含 189766 条记录（无标签）。

2 设计思路

各任务分别实现为独立的 Driver 类，共享 `Csv.java` 中的 CSV 解析辅助方法：缺失值 / 非法数值统一按 0 处理，表头通过首列是否为 `CUST_ID` 识别。

2.1 任务 1：各开户机构客户数量及资产统计

- **Mapper**：逐行解析 `train.csv`，以 `OPEN_ORG_NUM` 为 key，value 组合为 `1 <TAB> 资产 <TAB> 负债 <TAB> 是否逾期`；
- **Reducer**：按机构累加客户数、总资产、总负债、逾期客户数（`bad_good == 1`）。

输出格式：`机构编号 <TAB> 客户数量，总资产，总负债，个贷逾期客户数`。

2.2 任务 2：平均值分析

实现为两阶段流水线：

1. **Job A（map-only，单 reducer）**：读取任务 1 输出，计算每个机构的平均资产（总资产 / 客户数量），写入中间目录；
2. **Job B**：通过 `addCacheFile` 将平均值表加入 DistributedCache，Mapper 在 `setup()` 中加载为 `Map< 机构，平均资产 >`，遍历 `train.csv` 逐客户比较资产与机构平均值（等于平均值归入“高于”部分），Reducer 统计高于/低于数量。

2.3 任务 3：客户总资产四分位数

利用 MapReduce 框架内置的排序机制：Map 以 `CUST_ASSET_AMT` 为 key 输出，经 Shuffle（分区、排序）后进入单个 Reducer（`setNumReduceTasks(1)`），获得全局升序的资产序列；再结合总样本条数用线性插值法计算四分位数：

$$\text{pos} = p \cdot (n-1), \qquad Q_p = x_{\lfloor \text{pos} \rfloor} + (\text{pos} - \lfloor \text{pos} \rfloor) \cdot (x_{\lceil \text{pos} \rceil} - x_{\lfloor \text{pos} \rfloor})$$

该插值方式等价于 numpy 默认的 `linear` 插值。

2.4 任务 4：客户资产分层与资产结构统计

- **阈值注入**：Driver 先从任务 3 输出中解析 Q25/Q50/Q75，通过 `Configuration.setDouble` 注入作业；
- **分层**：Mapper 按资产与四分位数的关系把客户划分为低资产 / 中等资产 / 较高资产 / 高资产四层（tier 0–3），并对每层发射 7 类资产的金额；
- **聚合与排序**：Reducer 汇总每类资产金额后计算占比，先按层级升序（单 reducer 保证顺序集中），层内按占比降序排序输出。

2.5 任务 5：客户信贷风险预测（高斯朴素贝叶斯）

选用高斯朴素贝叶斯（Gaussian Naive Bayes）作为分类器，实现为“训练 + 预测”两个 Job：

1. **训练 Job**：TrainMapper 发射类别先验频度（key = `C<TAB>label`）与每类每特征的 `1<TAB>x<TAB>x`^{2}（key = `F<TAB>label<TAB>j`）；TrainReducer 汇总得到每类先验、均值、方差，输出紧凑模型文件（单 reducer）。
2. **预测 Job（map-only）**：模型经 DistributedCache 分发，PredictMapper 在 `setup()` 中加载模型，对每个测试样本计算两类的对数后验概率并取最大类别：

$$\hat{y} = \arg\max_{c} \left[\ln P(c) + \sum_{j=1}^{d} \ln \mathcal{N}(x_j; \mu_{c,j}, \sigma_{c,j}^2) \right]$$

针对训练集中某类下恒定特征（方差 ≈ 0，例如坏样本的 `G_OS_PRCP_SUM` 全为 0）会被高斯分布过度加权的问题，在 `setup()` 中对每个特征方差加入“该特征全局方差（组内 + 组间）的 1%”作为加性平滑项，避免单一零方差特征主导预测结果。

3　运行结果

全部作业在单机伪分布式集群上运行成功，共 7 个 MapReduce Job（任务 1、任务 3、任务 4 各 1 个，任务 2、任务 5 各 2 个）均为 SUCCEEDED 状态（见附录截图）。

3.1　任务 1 结果

共统计 687 个开户机构，节选如下（金额保留两位小数）：

表 3: 任务 1 结果节选

机构编号	客户数量	总资产	总负债	逾期客户数
1001	1254	184485852.19	112193577.37	32
1002	793	85762009.43	28727466.60	5
1003	887	116772867.93	23132764.24	9
101	1241	293699077.32	131735191.75	54

机构 1001：1254 个客户，总资产 184485852.19，总负债 112193577.37，逾期客户数 32 （即 `bad_good==1` 的客户数，经 `awk` 校验）。

3.2　任务 2 结果

共 687 个机构，节选如下：

表 4: 任务 2 结果节选

机构编号	高于平均资产客户数	低于平均资产客户数
1001	232	1022
1002	106	687
1003	144	743
101	203	1038

机构 1001 结果 `232,1022` 与示例一致。

3.3　任务 3 结果

四分位数输出（单行）：

```
Q25:1127.35    Q50:6501.38    Q75:49173.11
```

经 Python（numpy 线性插值）交叉验证一致。

3.4 任务 4 结果

共 28 行（4 层 × 7 类资产），完整结果如下：

表 5: 任务 4：各资产层级资产结构占比

等级	资产种类	占比
低资产	存款	0.9963
低资产	基金	0.0027
低资产	贵金属	0.0009
低资产	银保通	0.0001
低资产	国债	0.0000
低资产	理财	0.0000
低资产	私人理财	0.0000
中等资产	存款	0.9858
中等资产	基金	0.0102
中等资产	银保通	0.0029
中等资产	贵金属	0.0007
中等资产	国债	0.0003
中等资产	理财	0.0001
中等资产	私人理财	0.0000
较高资产	存款	0.9567
较高资产	基金	0.0267
较高资产	理财	0.0064
较高资产	银保通	0.0062
较高资产	国债	0.0032
较高资产	贵金属	0.0009
较高资产	私人理财	0.0000
高资产	存款	0.5602
高资产	理财	0.2949
高资产	私人理财	0.1086
高资产	基金	0.0265
高资产	国债	0.0083
高资产	银保通	0.0013
高资产	贵金属	0.0003

其中“中等资产 → 存款 → 0.9858”与示例一致。可以看出：低/中等资产客户以

存款为主（占比 > 98%），高资产客户资产配置显著多元化（理财 + 私人理财占比约 40%）。

3.5 任务 5 结果

对 189766 个测试客户完成预测，节选如下：

```
1000000239  0
1000003260  0
1000003414  0
1000007147  1
...
```

预测分布：预测为 0（好客户）184759 个，预测为 1（坏客户）5007 个，坏客户比例约 2.64%，与训练集先验（坏客户约 1.71%）基本一致，模型无退化。模型核心目标是掌握 MapReduce 分布式预测流程，分数不作为重点。

4 可能的改进之处

1. **去重 / 局部预聚合**：任务 1 中每个客户产生 1 条、任务 4 中每个客户产生 7 条中间记录，可添加 Combiner 在 Map 端先局部求和，显著减少 shuffle 数据量。
2. **任务 3 单 Reducer 内存风险**：四分位数计算依赖单 reducer 把全部资产加载进内存，数据量极大时可能 OOM，可改用近似分位数（如 t-digest）或多 reducer + 分桶汇总的方式。
3. **任务 4 依赖读取方式**：当前 Driver 用 `FileSystem.open` 读取任务 3 输出解析四分位数，可改用 DistributedCache 替代，使依赖关系在任务调度层面更规范。
4. **模型扩展性**：任务 5 目前为单 reducer 训练、内存加载模型，可扩展到多类别、加入特征选择 / 标准化；朴素贝叶斯假设特征独立，可尝试引入特征相关性处理或改用其他分类器。
5. **参数化与配置化**：特征索引、机构字段、输出格式等目前硬编码，可外置到配置文件或作业参数，便于复用。

附录：运行成功的 Web 页面截图

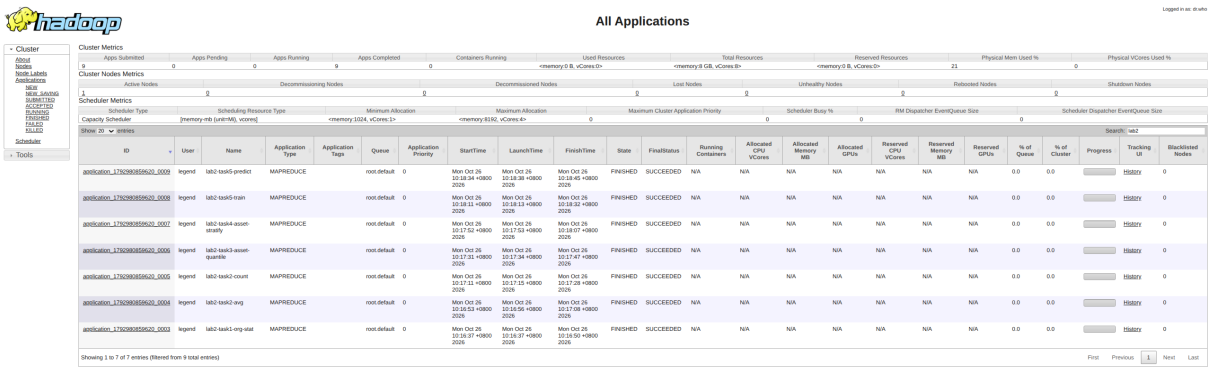


图 1: YARN ResourceManager 运行成功截图（全部作业 SUCCEEDED）